

Face Recognition Attendance System

Vision Document

Automated Attendance Using Deep Learning

Project Report

Prepared for College Presentation

1. Introduction

The Face Recognition Attendance System is an automated solution designed to replace traditional manual attendance methods such as paper registers, ID card swiping, or biometric fingerprint scanners. This system leverages deep learning-based face detection and recognition models to identify individuals in real-time using a standard webcam.

The project is built using Python with OpenCV's DNN module, utilizing ONNX-format models for high-performance inference. The YuNet model handles fast and accurate face detection, while the SFace model generates compact 128-dimensional face embeddings for recognition. A Tkinter-based graphical user interface provides an intuitive control panel for registering new faces, starting live recognition, and viewing attendance records.

Attendance data is automatically logged with name, date, time, and status into a CSV file, ensuring data persistence and easy export for further analysis. The system is designed to be lightweight, run on standard hardware without GPU acceleration, and suitable for classrooms, offices, and small organizations.

2. Problem Statement

Traditional attendance systems suffer from several limitations that affect accuracy, efficiency, and user experience:

- Manual paper-based attendance is time-consuming and prone to human errors.
- Proxy attendance (buddy punching) is common where one person marks attendance for another.
- RFID cards and fingerprint scanners require physical contact and dedicated hardware.
- Existing biometric systems are expensive to deploy and maintain.
- No real-time digital records - data must be manually digitized later.
- Difficult to generate reports or integrate with payroll/HR systems.

There is a clear need for a contactless, low-cost, and automated attendance system that prevents proxy attendance, provides instant digital records, and works with commodity hardware.

3. Proposed Solution

The proposed solution is a fully automated face recognition attendance system that operates through the following pipeline:

Webcam captures video frames -> Face detection (YuNet) locates faces in each frame -> Face recognition (SFace) extracts 128-dimensional feature vectors from aligned faces -> Feature vectors are compared against registered face database using cosine similarity -> If similarity exceeds threshold (0.30), identity is confirmed -> Attendance is automatically recorded with name, date, time, and 'Present' status -> Data is appended to CSV file -> Results are displayed in real-time on screen with bounding boxes and confidence scores.

The system includes a registration module (capture_photo.py) that allows new users to enroll by capturing their photo via webcam. The GUI (gui.py) provides a unified interface to start camera preview, register faces, launch live recognition, and view attendance records. All processing runs locally on CPU using ONNX models, requiring no internet connection or cloud services.

4. Project Objectives

- Develop a real-time face recognition system using YuNet and SFace ONNX models.
- Implement automated attendance marking with duplicate prevention (one entry per person per day).

- Build an intuitive Tkinter GUI for system control and monitoring.
- Store attendance records in CSV format for easy access and reporting.
- Support multiple registered users with dynamic face enrollment.
- Achieve reliable recognition accuracy on CPU without GPU dependency.
- Provide a modular, maintainable codebase with clear separation of concerns.

5. Technology Used

Programming Language & Core Libraries

- Python 3.x - Primary development language
- OpenCV (cv2) - Computer vision operations, DNN inference, camera handling
- NumPy - Numerical operations and array manipulation
- Tkinter - Native GUI framework for desktop application
- PIL/Pillow - Image processing for GUI display
- CSV module - Attendance data storage
- datetime - Timestamp generation
- os, sys, subprocess - System operations and process management

Deep Learning Models (ONNX Format)

- YuNet (face_detection_yunet_2026may.onnx) - Lightweight face detector optimized for CPU
- SFace (face_recognition_sface_2021dec.onnx) - Face recognition model producing 128-D embeddings

Both models are loaded via OpenCV's DNN module (cv2.FaceDetectorYN and cv2.FaceRecognizerSF), enabling fast inference without requiring deep learning frameworks like PyTorch or TensorFlow at runtime.

Data Storage

- CSV (attendance.csv) - Lightweight, human-readable attendance logs
- File system (known_faces/) - Registered face images stored as JPG files

6. System Architecture

The system follows a modular pipeline architecture with clear data flow between components:

Architecture Flow

Webcam -> Frame Capture -> Face Detection (YuNet) -> Face Alignment & Recognition (SFace) -> Feature Extraction (128-D Embedding) -> Identity Matching (Cosine Similarity) -> Attendance Recording -> CSV Storage -> GUI Display

Component Diagram

- GUI Layer (gui.py) - Main window, camera preview, control buttons
- Registration Module (capture_photo.py) - Captures and saves new face images
- Recognition Engine (live_test.py) - Core detection, recognition, attendance logic
- Model Layer - YuNet detector, SFace recognizer (ONNX models)
- Data Layer - known_faces/ directory, attendance.csv file

Data Flow Description

1. User launches gui.py which starts a camera preview using OpenCV and displays frames in a Tkinter label via PIL ImageTk conversion.
2. 'Register Face' button opens capture_photo.py as a subprocess, captures a photo, and saves it to known_faces/.
3. 'Live Recognition' button launches live_test.py which loads all known faces, extracts their embeddings, and starts the recognition loop.
4. Each frame: detector finds faces -> recognizer aligns and extracts features -> cosine similarity compares against known embeddings -> best match above threshold triggers attendance marking.
5. Attendance is written to CSV with duplicate check (one per person per day).
6. 'View Attendance' opens attendance.csv in default application.

7. Main Features

Face Registration

- One-click face enrollment via webcam (SPACE to capture, Q to cancel)
- Automatic naming from GUI input dialog
- Images saved to known_faces/ directory for persistence

Live Face Recognition

- Real-time detection and recognition at ~15-30 FPS on CPU
- Multi-face support - recognizes multiple people simultaneously
- Visual feedback: green box for known, red box for unknown
- Confidence score displayed on each detection
- Adjustable similarity threshold (default 0.30 cosine similarity)

Attendance Management

- Automatic logging: Name, Date (YYYY-MM-DD), Time (HH:MM:SS), Status
- Duplicate prevention - only first detection per person per day recorded
- CSV format compatible with Excel, Google Sheets, and data analysis tools
- Historical records preserved across sessions

User Interface

- Dark-themed modern Tkinter interface
- Live camera preview in main window
- Four primary actions: Register, Live Recognition, View Attendance, Exit
- Status messages and error handling with dialog boxes

8. Working Process

Step 1: System Initialization

The application starts with gui.py. The main window loads, camera preview begins (using cv2.VideoCapture with CAP_DSHOW for Windows), and frames are continuously converted from BGR to RGB, resized to 496x216, and displayed via PIL ImageTk.

Step 2: Face Registration

User clicks 'REGISTER FACE', enters name in dialog. System launches `capture_photo.py` with the name as argument. User positions face and presses SPACE. Photo is saved as `known_faces/<name>.jpg`. Camera preview resumes in GUI.

Step 3: Loading Known Faces

When 'LIVE RECOGNITION' is clicked, `live_test.py` loads all images from `known_faces/`, runs YuNet detection, SFace alignment and feature extraction for each, building an in-memory dictionary: `{name: 128-D embedding vector}`.

Step 4: Live Recognition Loop

For each webcam frame: `detector.setInputSize(frame_dims) -> detector.detect()` -> for each face: `recognizer.alignCrop()` -> `recognizer.feature()` -> cosine similarity match against all known embeddings -> best match selected -> if score $\geq$ 0.30: `mark_attendance(name)` -> draw bounding box and label.

Step 5: Attendance Recording

`mark_attendance()` checks if person already marked today (using in-memory set loaded from CSV at startup). If not, appends row to `attendance.csv` with current timestamp and updates the set. Console prints confirmation with details.

Step 6: Viewing Records

User clicks 'VIEW ATTENDANCE' which opens `attendance.csv` via `os.startfile()` - launching Excel, Notepad, or default CSV viewer on Windows.

9. Project Structure and Expected Output

Project Directory Structure

```
Face_Recognition/
|-- gui.py                # Main GUI application
|-- live_test.py          # Live recognition engine
|-- capture_photo.py      # Face registration module
|-- camera_test.py        # Simple camera test utility
|-- attendance.csv        # Attendance records (auto-generated)
|-- models/
|   |-- face_detection_yunet_2026may.onnx
|   |-- face_recognition_sface_2021dec.onnx
|-- known_faces/
|   |-- Shivani.jpg
|   |-- Tamanna.jpg
|   |-- ... (registered faces)
```

Expected Output Files

- `attendance.csv` - Grows with each recognized attendance event
- `known_faces/*.jpg` - One image per registered person
- Console output - Real-time logs of detections, matches, attendance marks

Sample attendance.csv Content

```
Name,Date,Time,Status
Tamanna,8/22/2026,13:04:07,Present
Shivani,8/22/2026,13:04:43,Present
```

Shivani, 2026-08-23, 17:25:20, Present
Tamanna, 2026-08-23, 17:41:28, Present
Shivani, 2026-08-24, 11:22:53, Present
Shivani, 2026-09-01, 09:45:06, Present

Live Recognition Window Output

- OpenCV window titled 'Face Recognition Attendance'
- Green bounding boxes around recognized faces with name and score
- Red bounding boxes around unknown faces
- Press 'Q' to exit recognition and return to GUI

10. Advantages, Limitations and Future Scope

Advantages

- Contactless - hygienic, no physical touch required
- Low cost - runs on any laptop/PC with webcam, no special hardware
- Anti-proxy - face recognition prevents buddy punching
- Real-time - instant attendance marking with visual confirmation
- Digital records - CSV format enables easy reporting and integration
- Offline operation - no internet/cloud dependency, data stays local
- Lightweight models - YuNet + SFace run efficiently on CPU
- Modular code - easy to extend, debug, and maintain
- Scalable - add unlimited users by simply capturing new photos

Limitations

- Lighting sensitivity - poor lighting reduces detection/recognition accuracy
- Angle sensitivity - extreme head poses may fail detection
- Single camera - no multi-camera support for large rooms
- No liveness detection - printed photos or screens could potentially spoof (basic version)
- Threshold tuning - fixed threshold (0.30) may need adjustment for different environments
- Windows-centric - uses CAP_DSHOW and os.startfile()
- No database - CSV can become slow with very large datasets
- GUI preview and recognition run separately (subprocess) - not fully integrated

Future Scope

- Add liveness detection (blink detection, depth analysis) to prevent spoofing
- Migrate to SQLite/PostgreSQL for better data management and querying
- Implement multi-camera support for large classrooms/halls
- Add web dashboard for remote attendance monitoring
- Integrate with student/employee management systems via API
- Support for face masks and partial occlusion handling
- Mobile app companion for admin notifications

- Advanced analytics: attendance trends, late arrivals, early departures
- Docker containerization for easy deployment
- Upgrade to newer models (e.g., InsightFace, RetinaFace) for higher accuracy

11. Conclusion

The Face Recognition Attendance System successfully demonstrates how modern deep learning techniques can be applied to solve real-world administrative problems. By combining the YuNet face detector and SFace recognizer - both optimized for CPU inference via ONNX - with a clean Python/OpenCV/Tkinter implementation, the system achieves reliable, real-time face recognition and automated attendance logging without requiring expensive hardware or cloud services.

Key achievements of this project include:

- End-to-end pipeline from face capture to CSV-based attendance records
- User-friendly GUI for non-technical operation
- Duplicate-prevention logic ensuring data integrity
- Modular architecture separating registration, recognition, and UI concerns
- Practical deployment on standard hardware with acceptable performance

While the current implementation has limitations regarding lighting conditions, spoofing resistance, and scalability, it provides a solid foundation for a production-grade system. The identified future enhancements - particularly liveness detection, database integration, and web-based monitoring - represent clear pathways to evolve this academic project into a robust enterprise solution.

This project illustrates the accessibility of computer vision technology today: with open-source models, standard libraries, and commodity hardware, students and developers can build functional AI applications that address genuine needs in educational and organizational settings.